REVISION CHEAT SHEET:

ASYNC / C#

async, await, and Task-Based Programming in C#

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

1. The Main Problem Async Solves
Async Version
2. Async Does Not Mean “Run on Another Thread”
Correct Mental Model
3. Async vs Concurrency vs Parallelism
Asynchrony
Concurrency
Parallelism
4. I/O-Bound vs CPU-Bound Work
I/O-Bound
CPU-Bound
5. What Is Task ?
Task Example
Await It
6. Task<T>
Awaiting Task
Example
7. async Modifier
Better
8. What await Actually Does
Important Fast Path
9. Execution Before the First Incomplete Await
10. Compiler-Generated Async State Machine
Conceptual Flow
11. Common Async Return Types
12. Task
13. Task<T>
14. async void
ASP.NET Core Warning
15. Async Method Naming
16. “Async All the Way”
17. Don't Use .Result
Don't Use .Wait()
Prefer
18. Why Blocking Async Is Dangerous
Thread Starvation
19. Classic Synchronization-Context Deadlock
20. If You Absolutely Must Block
21. Sequential Awaiting
22. Starting Independent Operations Concurrently
Retrieve Results
23. Concurrency Is Not Always Safe
Critical EF Core Rule
24. Task.WhenAll
Completion Rules
Task<T> Results
25. Task.WhenAny
Important
26. Exception Handling in Async Methods
Exception Flow
27. Exceptions Are Stored in the Task
28. Multiple Exceptions with WhenAll
29. Argument Validation and Async Methods
30. Cancellation Is Cooperative
31. CancellationToken
Cancellation Flow
32. Checking Cancellation
Manual Check
33. CancellationTokenSource
Timeout with CancelAfter
34. Linked Cancellation Tokens
35. ASP.NET Core Request Cancellation
36. Do Not Swallow Cancellation

37. Sequential vs Concurrent Work
Concurrent
But Only If Independent
38. Task.Delay vs Thread.Sleep
39. Task.Run
Don't Wrap I/O in Task.Run
ASP.NET Core and Task.Run
40. Fire-and-Forget
Especially Dangerous in ASP.NET Core
41. ConfigureAwait
Why Does This Matter?
ASP.NET Core
Important
42. ValueTask<T>
Don't Use ValueTask Everywhere
Important ValueTask Constraints

ASYNC / C#

Topic 27 of 40

Asynchronous programming lets an application begin an operation that may take time, avoid blocking a thread while that operation is waiting, and continue when the operation completes.

Typical asynchronous operations include:

EXAMPLE / FLOW

```
Database queries

HTTP requests

File I/O

Network calls

Message-broker operations

Delays and timers
```

In modern .NET, the primary asynchronous programming model is the **Task-based Asynchronous Pattern (TAP)**. `Task` and `Task<T>` represent asynchronous operations, while C# provides `async` and `await` as language-level support for composing those operations. Microsoft recommends TAP for new .NET asynchronous APIs.

1. The Main Problem Async Solves

Suppose a Web API calls a database:

C#

```
Gift? gift =
    dbContext.Gifts
        .SingleOrDefault(
            gift =>
                gift.GiftId
                == giftId);
```

During database I/O, the application may spend substantial time simply:

EXAMPLE / FLOW

```
Waiting
```

The thread has no useful CPU work to perform while the database is processing the request and transferring the response.

A synchronous model behaves conceptually like:

EXAMPLE / FLOW

ASYNC / C#

```
Thread
    ↓
Send database request
    ↓
WAIT
WAIT
WAIT
WAIT
    ↓
Database response
    ↓
Continue
```

The thread remains occupied during the wait.

Async Version

C#

```
Gift? gift =
    await dbContext.Gifts
        .SingleOrDefaultAsync(
            gift =>
                gift.GiftId
                == giftId, cancellationToken);
```

Conceptually:

EXAMPLE / FLOW

```
Thread
    ↓
Start database operation
    ↓
Operation not complete
    ↓
Return thread for other useful work
    ↓
Database eventually completes
    ↓
Continuation scheduled
    ↓
Method resumes
```

The important benefit is:

> **The thread doesn't need to sit blocked while I/O is pending.**

ASP.NET Core specifically recommends asynchronous I/O because blocking request threads can contribute to thread-pool starvation and reduced throughput.

2. Async Does Not Mean “Run on Another Thread”

This misconception is extremely common.

Consider:

C#

```
string json = await httpClient.GetStringAsync(url, cancellationToken);
```

This does **not** mean:

ASYNC / C#

EXAMPLE / FLOW

```
Create another thread
    ↓
Make that thread sit waiting
```

For true asynchronous I/O, the operating system or underlying I/O infrastructure can track the operation while no application thread remains blocked.

Microsoft explicitly notes that `async` and `await` don't automatically create additional threads.

Correct Mental Model

EXAMPLE / FLOW

```
async / await
    ≠
new thread
```

Instead:

EXAMPLE / FLOW

```
async / await
    =
Compose operations that may
complete asynchronously
without blocking a thread
```

3. Async vs Concurrency vs Parallelism

These concepts are related but different.

Asynchrony

Concerned mainly with:

EXAMPLE / FLOW

```
Waiting efficiently
```

Example:

EXAMPLE / FLOW

```
Database query
HTTP request
File read
```

Concurrency

Means:

EXAMPLE / FLOW

```
Multiple operations are
in progress during overlapping time
```

Example:

EXAMPLE / FLOW

```
HTTP request A
HTTP request B
HTTP request C
```

ASYNC / C#

may all be waiting concurrently.

Parallelism

Means:

EXAMPLE / FLOW

```
Multiple operations are
actually executing simultaneously
```

usually on multiple CPU cores.

Example:

EXAMPLE / FLOW

```
CPU core 1 → calculation A

CPU core 2 → calculation B
```

Parallelism and threading are covered deeply in **Topic 28**.

For now:

EXAMPLE / FLOW

```
I/O-bound work
    →
async / await

CPU-bound parallel work
    →
threads / Task.Run / parallel APIs
depending on scenario
```

4. I/O-Bound vs CPU-Bound Work

This distinction is critical.

I/O-Bound

The program spends most of its time waiting for something outside the CPU:

EXAMPLE / FLOW

```
SQL Server

HTTP API

Disk

Network

Message queue
```

Typical implementation:

C#

```
await SomeIoOperationAsync();
```

You generally don't need:

ASYNC / C#

```
C#
Task.Run(...)
```

around naturally asynchronous I/O.

Microsoft explicitly recommends using `async` / `await` directly for I/O-bound operations rather than wrapping them in `Task.Run`.

CPU-Bound

The CPU is actively computing:

EXAMPLE / FLOW

```
Image processing

Compression

Large mathematical calculation

Encryption

Large in-memory transformation
```

A desktop/UI application might use:

```
C#
await Task.Run(
    () =>
        PerformExpensiveCalculation());
```

to keep the UI thread responsive.

But in ASP.NET Core, putting CPU-heavy request work into `Task.Run` doesn't magically make the server more scalable. It still consumes a thread-pool thread.

We'll study this more in Topic 28.

5. What Is Task?

A:

```
C#
Task
```

represents an operation whose completion may occur:

EXAMPLE / FLOW

```
Now

Later

Successfully

With an exception

Through cancellation
```

A `Task` isn't necessarily a thread.

Think of it as:

ASYNC / C#

An object representing the eventual completion of an operation.

Task Example

C#
```
Task task = SendReceiptAsync(giftId, cancellationToken);
```

At this point:

EXAMPLE / FLOW
```
task
```

represents:

EXAMPLE / FLOW
```
"SendReceiptAsync is in progress
or perhaps already complete."
```

Await It

C#
```
await task;
```

means:

EXAMPLE / FLOW
```
Do not continue this method
past this point
until task has completed.
```

But importantly:

EXAMPLE / FLOW
```
Do not block the current thread
while waiting if suspension is required.
```

6. Task<T>

When an asynchronous operation eventually produces a value:

C#
```
Task<T>
```

represents that future result.

Example:

C#
```
Task<Gift?> task = GetGiftAsync(giftId, cancellationToken);
```

Eventually the task may produce:

EXAMPLE / FLOW

```
Gift

or null
```

ASYNC / C#

Awaiting Task

C#

```
Gift? gift = await task;
```

You can think of:

EXAMPLE / FLOW

```
Task<Gift?>
        ↓
await
        ↓
Gift?
```

Example

C#

```
public async Task<Gift?>
    GetGiftAsync(
        long giftId,
        CancellationToken cancellationToken)
{
    Gift? gift =
        await _dbContext.Gifts
            .SingleOrDefaultAsync(
                gift =>
                    gift.GiftId
                    == giftId, cancellationToken);

    return gift;
}
```

7. async Modifier

A method can be declared:

C#

```
public async Task ProcessAsync()
{
}
```

The `async` modifier enables features such as:

C#

```
await
```

inside the method.

But:

Marking a method `async` doesn't automatically make its work asynchronous.

This method:

ASYNC / C#

```
C#
public async Task BadAsync()
{
    Thread.Sleep(
        5000);
}
```

still blocks the thread.

The compiler will also warn that the method contains no `await`.

Better

If the operation is truly an asynchronous delay:

```
C#
public async Task BetterAsync()
{
    await Task.Delay(
        5000);
}
```

Microsoft recommends replacing blocking operations such as `Thread.Sleep`, `Task.Wait`, and `.Result` with asynchronous equivalents such as `Task.Delay` and `await` when appropriate.

8. What `await` Actually Does

Consider:

```
C#
Gift gift = await GetGiftAsync();
```

Conceptually, `await` does something like:

EXAMPLE / FLOW

```
Is the awaited operation already complete?
        ↓
Yes
    →
Continue immediately
```

If not:

EXAMPLE / FLOW

```
Register the remainder of this method
as a continuation

Return control to the caller

Resume later when task completes
```

Microsoft explains that an `await` on an incomplete operation suspends the rest of the method and registers it as a continuation rather than blocking the thread.

Important Fast Path

If the task is already completed when reached:

```
C#
await task;
```

the method may simply continue synchronously.

Therefore:

```
EXAMPLE / FLOW
await
```

does **not** guarantee:

```
EXAMPLE / FLOW
Thread switch

Context switch

Yield
```

9. Execution Before the First Incomplete Await

Consider:

```
C#
public async Task ProcessAsync()
{
    Console.WriteLine(
        "A");

    await SomeOperationAsync();

    Console.WriteLine(
        "B");
}
```

When called:

```
C#
Task task = ProcessAsync();
```

the method begins running immediately.

Conceptually:

```
EXAMPLE / FLOW
Call ProcessAsync
    ↓
Print A
    ↓
Call SomeOperationAsync
    ↓
Awaited task incomplete?
```

If yes:

```
EXAMPLE / FLOW
ProcessAsync returns its Task
to caller
```

Later:

ASYNC / C#

EXAMPLE / FLOW

```
SomeOperationAsync completes
    ↓
ProcessAsync resumes
    ↓
Print B
```

An async method typically returns control when it encounters the first awaited operation that isn't yet complete, or when it reaches the end.

10. Compiler-Generated Async State Machine

Async methods look sequential:

C#

```
Gift gift = await LoadGiftAsync();

await ValidateAsync(
    gift);

await SaveAsync(
    gift);
```

But the compiler transforms the method into a state-machine-like structure capable of:

EXAMPLE / FLOW

```
Remembering local variables

Remembering where execution paused

Registering continuations

Resuming after completion

Storing results

Storing exceptions
```

You don't normally write this state machine yourself.

`async` / `await` lets you write code that appears sequential while the compiler performs the continuation plumbing underneath.

Conceptual Flow

EXAMPLE / FLOW

```
State 0
    ↓
Start method
    ↓
await LoadGiftAsync
    ↓
suspend
```

Later:

EXAMPLE / FLOW

```
State 1
    ↓
continue after Load
    ↓
await ValidateAsync
    ↓
suspend
```

ASYNC / C#

Later:

EXAMPLE / FLOW

```
State 2
    ↓
continue after Validate
    ↓
await SaveAsync
```

11. Common Async Return Types

The main async return types are:

EXAMPLE / FLOW

```
Task

Task<T>

void

ValueTask

ValueTask<T>

IAsyncEnumerable<T>
```

C# async methods also support custom awaitable return types that satisfy the required awaiter pattern.

12. Task

Use when:

EXAMPLE / FLOW

```
The method performs asynchronous work

but

does not produce a result value
```

Example:

C#

```
public async Task SendReceiptAsync(
    long giftId,
    CancellationToken cancellationToken)
{
    await _receiptClient.SendAsync(
        giftId,
        cancellationToken);
}
```

13. Task<T>

ASYNC / C#

Use when the operation returns a value.

```
C#
public async Task<Gift?>
    FindGiftAsync(
        long giftId,
        CancellationToken cancellationToken)
{
    return await _dbContext.Gifts
        .SingleOrDefaultAsync(
            gift =>
                gift.GiftId
                == giftId, cancellationToken);
}
```

14. async void

Avoid:

```
C#
public async void SaveGiftAsync()
{
}
```

except for event handlers whose delegate signature requires `void`.

Why?

An `async void` method:

```
EXAMPLE / FLOW
Cannot be awaited

Does not return a Task to represent completion

Makes exception handling by callers difficult

Makes composition difficult
```

Microsoft recommends `Task` instead of `void` for asynchronous methods except event handlers.

ASP.NET Core Warning

Controller actions should not be:

```
C#
public async void Save()
```

They should return something task-based:

```
C#
public async Task<IActionResult>
    Save()
```

ASP.NET Core specifically warns against `async void` because the request can appear complete before asynchronous work has actually finished.

15. Async Method Naming

ASYNC / C#

The standard TAP naming convention is:

EXAMPLE / FLOW

```
MethodNameAsync
```

Examples:

C#

```
GetGiftAsync

SaveChangesAsync

SendAsync

ReadAsync
```

Use the `Async` suffix for asynchronous methods returning awaitable types, particularly when a synchronous counterpart exists or when naming clarity benefits callers. Microsoft documents this as part of TAP API design guidance.

16. “Async All the Way”

Suppose:

EXAMPLE / FLOW

```
Controller
    ↓
Service
    ↓
Repository
    ↓
EF Core
```

If the database operation is async:

C#

```
await dbContext.Gifts
    .ToListAsync();
```

prefer making the entire call chain asynchronous:

EXAMPLE / FLOW

```
Controller async
    ↓
Service async
    ↓
Repository async
    ↓
Database async
```

instead of:

EXAMPLE / FLOW

```
Async repository
    ↓
.Result
```

```
    ↓
Sync service
```

ASYNC / C#

Microsoft recommends using async throughout the call stack where possible rather than synchronously blocking on asynchronous work.

17. Don't Use .Result

Bad:

C#

```
Gift? gift = GetGiftAsync(giftId, cancellationToken).Result;
```

This:

EXAMPLE / FLOW

```
Blocks the current thread
```

until the task completes.

Don't Use .Wait()

Bad:

C#

```
GetGiftAsync(
        giftId,
        cancellationToken)
    .Wait();
```

Also blocks.

Prefer

C#

```
Gift? gift = await GetGiftAsync(giftId, cancellationToken);
```

Microsoft's async guidance explicitly recommends replacing `.Result`, `.Wait()`, `WaitAll`, and `WaitAny` with their await-based equivalents.

18. Why Blocking Async Is Dangerous

Blocking on asynchronous work can cause two major problems:

EXAMPLE / FLOW

```
Thread starvation

Deadlocks in context-bound environments
```

Thread Starvation

ASP.NET Core handles many requests using a shared thread pool.

Imagine:

EXAMPLE / FLOW

```
100 requests

Each thread calls .Result

All 100 threads block waiting for I/O
```

ASYNC / C#

Now the runtime may need additional threads to process work that would have completed those requests.

This can create:

EXAMPLE / FLOW

```
Reduced throughput

High latency

Thread-pool growth

Thread starvation
```

ASP.NET Core performance guidance strongly recommends avoiding blocking calls.

19. Classic Synchronization-Context Deadlock

A typical UI deadlock looks conceptually like:

EXAMPLE / FLOW

```
UI thread
    ↓
Call async method
    ↓
async method awaits I/O
    ↓
UI thread calls .Result
    ↓
UI thread blocks
```

Later:

EXAMPLE / FLOW

```
I/O completes
    ↓
continuation wants UI context
    ↓
UI thread is blocked waiting
for the continuation
```

Result:

EXAMPLE / FLOW

```
Deadlock
```

This is one reason sync-over-async is dangerous.

ASP.NET Core doesn't install the classic ASP.NET request `SynchronizationContext` by default, so this specific context deadlock pattern is less common there; blocking is still harmful because it wastes request-processing threads and can contribute to starvation.

20. If You Absolutely Must Block

ASYNC / C#

Occasionally legacy synchronous code cannot yet be changed.

Microsoft's current guidance lists:

```
C#
task.GetAwaiter()
    .GetResult();
```

as preferable to `.Wait()` or `.Result` when synchronous blocking is unavoidable because it preserves the original exception rather than wrapping it in `AggregateException`.

However:

> **It still blocks and still carries deadlock/starvation risks.**

The preferred solution remains async all the way.

21. Sequential Awaiting

Consider:

```
C#
Gift gift = await LoadGiftAsync();

Campaign campaign = await LoadCampaignAsync();

Fund fund = await LoadFundAsync();
```

This means:

```
EXAMPLE / FLOW
Load Gift
    ↓
wait until complete
    ↓
Load Campaign
    ↓
wait
    ↓
Load Fund
```

If each takes one second:

```
EXAMPLE / FLOW
~3 seconds
```

when operations are truly independent.

22. Starting Independent Operations Concurrently

If the three operations are independent:

```
C#
Task<Gift> giftTask = LoadGiftAsync();

Task<Campaign> campaignTask = LoadCampaignAsync();
```

```
Task<Fund> fundTask = LoadFundAsync();

await Task.WhenAll(
    giftTask,
    campaignTask,
    fundTask);
```

ASYNC / C#

Now they are all started before you wait for all of them.

Conceptually:

EXAMPLE / FLOW

```
Start Gift ──────────┐
Start Campaign ──────┼── overlapping wait
Start Fund ──────────┘
                     ↓
               WhenAll complete
```

Microsoft recommends starting independent asynchronous work before awaiting when concurrency is desired.

Retrieve Results

C#

```
Gift gift = await giftTask;

Campaign campaign = await campaignTask;

Fund fund = await fundTask;
```

Since `WhenAll` has already completed successfully, these awaits complete immediately.

You can also use:

C#

```
var results = await Task.WhenAll(task1, task2, task3);
```

when all tasks return the same `T`.

23. Concurrency Is Not Always Safe

Do not blindly turn:

C#

```
await OperationAAsync();

await OperationBAsync();
```

into concurrent tasks.

Ask:

EXAMPLE / FLOW

```
Are operations independent?

Do they share mutable state?

Do they use the same DbContext?

Does operation B depend on A?
```

Can the downstream system handle concurrency?

ASYNC / C#

Critical EF Core Rule

EF Core does **not** support multiple parallel operations on the same `DbContext` instance.

This is unsafe:

C#

```
Task<Gift?> giftTask = _dbContext.Gifts.SingleOrDefaultAsync(gift => gift.GiftId == giftId);

Task<Campaign?> campaignTask =
    _dbContext.Campaigns
        .SingleOrDefaultAsync(
            campaign =>
                campaign.CampaignId
                == campaignId);

await Task.WhenAll(
    giftTask,
    campaignTask);
```

if both operations use the same context concurrently.

EF Core explicitly requires awaiting one operation before beginning another on the same `DbContext`, or using separate contexts for truly parallel operations.

24. Task.WhenAll

`Task.WhenAll` returns a task that completes when **all supplied tasks** complete.

C#

```
await Task.WhenAll(
    firstTask,
    secondTask,
    thirdTask);
```

It does not synchronously block the calling thread.

Completion Rules

If all tasks succeed:

EXAMPLE / FLOW

```
WhenAll
    →
RanToCompletion
```

If any task faults:

EXAMPLE / FLOW

```
WhenAll
    →
Faulted
```

If none fault, but one or more are canceled:

EXAMPLE / FLOW

```
WhenAll
    →
Canceled
```

ASYNC / C#

Microsoft documents these status rules explicitly.

Task<T> Results

C#

```
Task<string>[] tasks = [GetAAsync(), GetBAsync(), GetCAsync()];

string[] results = await Task.WhenAll(tasks);
```

The resulting array preserves the input task order, not completion order.

25. Task.WhenAny

`Task.WhenAny` completes when the **first supplied task finishes**, whether that task:

EXAMPLE / FLOW

```
Succeeded

Faulted

Was canceled
```

Example:

C#

```
Task<Gift> completed = await Task.WhenAny(primaryTask, backupTask);
```

Then:

C#

```
Gift gift = await completed;
```

The second await matters because `WhenAny` tells you which task completed; awaiting that actual task retrieves its result or propagates its exception.

Important

The tasks that didn't win:

EXAMPLE / FLOW

```
continue running
```

unless you cancel them yourself.

`WhenAny` does not automatically cancel the losing tasks.

26. Exception Handling in Async Methods

Normal `try/catch` syntax works naturally:

```C#
try
{
    Gift gift = await LoadGiftAsync(cancellationToken);
}
catch (
    HttpRequestException ex)
{
    // Handle failure.
}
```

ASYNC / C#

If asynchronous work throws, the returned task becomes faulted.

When awaited, the exception is rethrown at the await point so you can handle it like synchronous code.

Exception Flow

EXAMPLE / FLOW

```
Async operation
    ↓
throws
    ↓
Task becomes Faulted
    ↓
Caller awaits Task
    ↓
Exception rethrown
    ↓
catch executes
```

27. Exceptions Are Stored in the Task

Suppose:

```C#
public async Task ProcessAsync()
{
    await Task.Delay(
        100);

    throw new InvalidOperationException(
        "Processing failed.");
}
```

Calling:

```C#
Task task = ProcessAsync();
```

returns a Task.

Later:

```C#
await task;
```

surfaces the failure.

`Task.Exception` stores an `AggregateException`, while `await` normally rethrows the relevant underlying exception directly so asynchronous code behaves more naturally.

28. Multiple Exceptions with `WhenAll`

ASYNC / C#

Suppose:

```C#
Task first = FailOneAsync();

Task second = FailTwoAsync();

Task combined = Task.WhenAll(first, second);
```

Then:

```C#
try
{
    await combined;
}
catch (
    Exception ex)
{
}
```

The combined task records all failures in:

```C#
combined.Exception
```

which is an `AggregateException`.

Awaiting the faulted task typically throws one underlying exception directly rather than throwing that aggregate wrapper. If you need to inspect every failure, examine `combined.Exception.InnerExceptions`.

29. Argument Validation and Async Methods

There is a subtle distinction here.

An exception thrown **inside an `async` method body** is represented through the returned Task.

If public API argument validation should fail synchronously, Microsoft recommends using a non-async outer method that validates arguments and then calls an inner async implementation.

Example:

```C#
public Task<Gift>
    LoadGiftAsync(
        long giftId,
        CancellationToken cancellationToken)
{
    if (giftId <= 0)
    {
        throw new ArgumentOutOfRangeException(
            nameof(giftId));
    }

    return LoadGiftCoreAsync(
        giftId,
        cancellationToken);
}

private async Task<Gift>
```

```
    LoadGiftCoreAsync(
        long giftId,
        CancellationToken cancellationToken)
{
    Gift? gift =
        await _dbContext.Gifts
            .SingleOrDefaultAsync(
                gift =>
                    gift.GiftId
                    == giftId, cancellationToken);

    return gift
        ??
        throw new InvalidOperationException(
            "Gift not found.");
}
```

ASYNC / C#

30. Cancellation Is Cooperative

Cancellation in .NET doesn't normally mean:

```
EXAMPLE / FLOW
Forcefully kill operation immediately.
```

Instead:

```
EXAMPLE / FLOW
Caller requests cancellation

Operation observes request

Operation stops at a safe point
```

This is called:

```
EXAMPLE / FLOW
Cooperative cancellation
```

`CancellationToken` is the standard mechanism for this model.

31. CancellationToken

Typical method:

```
C#
public async Task<Gift?>
    GetGiftAsync(
        long giftId,
        CancellationToken cancellationToken)
{
    return await _dbContext.Gifts
        .SingleOrDefaultAsync(
            gift =>
                gift.GiftId
                == giftId, cancellationToken);
}
```

Notice the token is:

EXAMPLE / FLOW

```
Accepted

then forwarded
```

ASYNC / C#

to the underlying async operation.

Cancellation Flow

EXAMPLE / FLOW

```
Caller
    ↓
CancellationToken
    ↓
Service
    ↓
Repository / EF Core
    ↓
Database provider
```

This is often called:

EXAMPLE / FLOW

```
Propagating cancellation
```

32. Checking Cancellation

For custom loops:

C#

```
for (
    int i = 0;
    i < items.Count;
    i++)
{
    cancellationToken
        .ThrowIfCancellationRequested();

    Process(
        items[i]);
}
```

If cancellation has been requested:

C#

```
ThrowIfCancellationRequested()
```

throws:

EXAMPLE / FLOW

```
OperationCanceledException
```

with the token.

This is the standard way for many custom operations to report cooperative cancellation.

Manual Check

ASYNC / C#

You can also write:

```C#
if (
    cancellationToken
        .IsCancellationRequested)
{
    return;
}
```

But if callers need the Task to be marked:

EXAMPLE / FLOW

```
Canceled
```

rather than simply:

EXAMPLE / FLOW

```
RanToCompletion
```

`ThrowIfCancellationRequested()` is often the more semantically appropriate approach.

33. CancellationTokenSource

A:

```C#
CancellationTokenSource
```

owns the ability to request cancellation.

Example:

```C#
using CancellationTokenSource cts = new();

CancellationToken token = cts.Token;
```

Pass:

```C#
token
```

to consumers.

Request cancellation:

```C#
cts.Cancel();
```

The token itself doesn't initiate cancellation; the source does.

Timeout with CancelAfter

ASYNC / C#

```
C#
using CancellationTokenSource cts = new();

cts.CancelAfter(
    TimeSpan.FromSeconds(
        10));

await ProcessAsync(
    cts.Token);
```

If processing exceeds the timeout:

```
EXAMPLE / FLOW
Cancellation requested
```

Microsoft documents `CancelAfter` as a standard way to schedule cancellation after a timeout.

34. Linked Cancellation Tokens

Sometimes an operation should stop because:

```
EXAMPLE / FLOW
Caller canceled

OR

Internal timeout occurred
```

Use:

```
C#
using CancellationTokenSource timeoutCts = new(TimeSpan.FromSeconds(10));

using CancellationTokenSource linkedCts =
    CancellationTokenSource
        .CreateLinkedTokenSource(
            callerToken,
            timeoutCts.Token);
```

Then:

```
C#
await OperationAsync(
    linkedCts.Token);
```

Either source can request cancellation.

35. ASP.NET Core Request Cancellation

ASP.NET Core exposes:

```
C#
HttpContext.RequestAborted
```

as a `CancellationToken`.

Controller actions can usually receive it directly:

ASYNC / C#

```C#
[HttpGet("{giftId:long}")]
public async Task<ActionResult<GiftResponse>>
    GetGift(
        long giftId,
        CancellationToken cancellationToken)
{
    Gift? gift = await _service.GetGiftAsync(giftId, cancellationToken);

    // ...
}
```

If the client disconnects or the request is aborted, that token can signal downstream work to stop.

Microsoft recommends propagating `RequestAborted` into long-running database/HTTP operations where appropriate.

36. Do Not Swallow Cancellation

Bad:

```C#
try
{
    await ProcessAsync(
        cancellationToken);
}
catch (
    Exception)
{
    return;
}
```

This treats:

EXAMPLE / FLOW

```
Cancellation

and

Actual failure
```

as the same thing.

Better:

```C#
try
{
    await ProcessAsync(
        cancellationToken);
}
catch (
    OperationCanceledException)
    when (
        cancellationToken
            .IsCancellationRequested)
{
```

```
    throw;
}
```

ASYNC / C#

or simply allow cancellation to propagate when no special handling is needed.

37. Sequential vs Concurrent Work

Suppose receipt creation and analytics lookup are independent.

Sequential:

```
C#
Receipt receipt = await BuildReceiptAsync(gift);

AnalyticsSummary summary = await LoadAnalyticsAsync(gift);
```

Timeline:

```
EXAMPLE / FLOW
Receipt ──────────
                Analytics ──────────
```

Concurrent

```
C#
Task<Receipt> receiptTask = BuildReceiptAsync(gift);

Task<AnalyticsSummary> analyticsTask = LoadAnalyticsAsync(gift);

await Task.WhenAll(
    receiptTask,
    analyticsTask);

Receipt receipt = await receiptTask;

AnalyticsSummary summary = await analyticsTask;
```

Timeline:

```
EXAMPLE / FLOW
Receipt   ──────────────
Analytics ──────────
```

Total elapsed time can be closer to the slower operation rather than their sum.

But Only If Independent

Don't do:

```
C#
Task<Gift> loadTask = LoadGiftAsync();

Task saveTask = SaveGiftAsync();
```

if:

```
EXAMPLE / FLOW
SaveGiftAsync needs the loaded Gift
```

Dependency means sequential ordering is required.

ASYNC / C#

38. Task.Delay vs Thread.Sleep

Blocking:

```
C#
Thread.Sleep(
    5000);
```

The thread cannot perform other work for five seconds.

Async delay:

```
C#
await Task.Delay(
    5000,
    cancellationToken);
```

The method can suspend without keeping that thread blocked.

Microsoft explicitly recommends `Task.Delay` over `Thread.Sleep` in async flows.

39. Task.Run

`Task.Run` queues work to the thread pool.

Example:

```
C#
decimal result = await Task.Run(() => CalculateExpensiveReport());
```

It can be appropriate for CPU-intensive work in a client application where you need to free a UI thread.

Don't Wrap I/O in Task.Run

Bad:

```
C#
await Task.Run(
    async () =>
        await httpClient.GetAsync(
            url));
```

`HttpClient.GetAsync` is already asynchronous.

The extra `Task.Run`:

```
EXAMPLE / FLOW
Adds scheduling overhead

Uses thread-pool resources unnecessarily

Doesn't make the I/O "more async"
```

For I/O-bound work, use the async API directly.

ASP.NET Core and Task.Run

Avoid assuming:

C#

```
Task.Run(...)
```

ASYNC / C#

is a way to create safe background work from a request.

ASP.NET Core recommends using hosted/background services for durable or long-running background work rather than capturing request state and launching unmanaged fire-and-forget tasks.

40. Fire-and-Forget

Dangerous:

C#

```
_ = SendReceiptAsync(giftId, cancellationToken);
```

Now:

EXAMPLE / FLOW

```
Caller doesn't wait

Completion isn't coordinated

Exception may go unobserved

Request-scoped dependencies may disappear

Cancellation lifetime may be wrong
```

Especially Dangerous in ASP.NET Core

C#

```
_ =
    Task.Run(
        async () =>
        {
            await _dbContext
                .SaveChangesAsync();
        });
```

The request may end.

The scoped:

EXAMPLE / FLOW

```
DbContext
```

may be disposed.

ASP.NET Core recommends hosted services/background queues for work that should continue after a request.

41. ConfigureAwait

A normal:

C#

```
await task;
```

may capture an available:

```
EXAMPLE / FLOW
SynchronizationContext

or custom TaskScheduler
```

so the continuation can resume through that context.

```
C#
await task.ConfigureAwait(
    false);
```

tells the awaiter not to force the continuation back to the captured context.

Why Does This Matter?

In desktop UI applications:

```
EXAMPLE / FLOW
UI thread
```

may need continuations that update UI controls to resume on that UI context.

General-purpose library code often does not care where the continuation runs.

ASP.NET Core

ASP.NET Core does not install the classic ASP.NET `SynchronizationContext` by default.

Therefore application code in ASP.NET Core usually doesn't need to sprinkle:

```
C#
.ConfigureAwait(false)
```

everywhere merely to avoid the classic ASP.NET deadlock.

Microsoft's guidance is generally:

```
EXAMPLE / FLOW
Application code
    →
usually use normal await

General-purpose library code
    →
consider ConfigureAwait(false)
when context independence is intended
```

Important

`ConfigureAwait(false)` does **not** mean:

```
EXAMPLE / FLOW
Continue on a different thread
```

It means:

EXAMPLE / FLOW

```
Do not require the continuation
to marshal back to the captured context.
```

ASYNC / C#

If the Task is already complete, code may simply continue synchronously on the current thread.

42. ValueTask<T>

A:

C#

```
ValueTask<T>
```

is a value-type awaitable.

It can reduce Task allocation overhead in specialized scenarios where operations very frequently complete synchronously.

Example signature:

C#

```
public ValueTask<Gift?>
    TryGetCachedGiftAsync(
        long giftId)
```

Don't Use ValueTask Everywhere

`Task<T>` should remain the normal default.

Microsoft's TAP guidance recommends `ValueTask` mainly where measurement demonstrates allocation pressure and callers can handle its usage constraints.

Important ValueTask Constraints

A `ValueTask<T>` may represent different underlying mechanisms and has stricter consumption rules.

In general, don't:

EXAMPLE / FLOW

```
Await
```